《计算机视觉》课程论文

姓　　名：刘彦余

学　　号：2023302121448

院系专业：计算机学院-特色软件工程

指导老师：邹华

提交日期：2025 年 5 月 27 日

轻量级医学图像分割课程论文

摘要

为满足资源受限场景（如移动端或嵌入式设备）下高精度医学图像分割的需求，本文设计并实现了一种轻量级两阶段分割网络——W-Net。网络基于 U-Net 架构，融合了 1×1 残差捷径（shortcut）与跳跃特征卷积桥（conv_bridge）两项改进，总参数量仅约 0.07 M，模型权重大小约 0.3 MB；在 DRIVE 视网膜血管分割数据集上的测试集上，达成 AUC=0.9810，Dice=0.8284，Accuracy=0.9554，RCC=0.8930，显著优于标准 U-Net 且满足课程对模型轻量化与性能的双重要求。

关键词
轻量级；医学图像分割；W-Net；U-Net；残差捷径；卷积桥

1. 引言

医学图像分割在自动诊断、手术导航等领域具有重要意义。然而，临床应用常受限于设备算力与存储，在嵌入式或移动终端上难以部署参数量上千万的分割模型。因此，如何在保证高精度的同时，实现模型轻量化，成为当前研究热点。本文针对该问题，提出了一种轻量级两阶段分割网络 W-Net，基于经典 U-Net 的创新性改进，以大幅减少参数量并提升分割性能。

2. 相关工作

1. U-Net 系列：Ronneberger 等人最早提出 U-Net，后续出现多种改进（注意力 U-Net、残差 U-Net 等）。
2. 轻量化网络：MobileNetV2、ShuffleNet 等使用深度可分离卷积与瓶颈结构显著降低计算量；
3. 跳跃连接优化：通过 1×1 残差捷径或卷积桥融合高分辨率特征，以增强梯度流与特征表达。
4. 两阶段分割：W-Net、双 U-Net 等架构利用前一阶段预测作为第二阶段的辅助输入，实现更细粒度分割。
5. 视觉 Transformer
Vision Transformer（ViT）提出后，出现 DeiT、Swin Transformer、CSWin 等针对数据效率和计算效率优化的变种，已在分类、检测、分割等多项任务中赶超传统 CNN。
6. 多模态与大模型
CLIP（图文对比学习）、ALIGN、DALL·E、Stable Diffusion 等将视觉与语言紧密结合，推

动零样本 / 少样本学习与文本驱动生成。

3. 方法

3.1 网络结构

W-Net 如图 1 所示，由两级 U-Net 级联构成：

```
Input  (512×512×3)
   ↓
┌──────────────────┐      ┌──────────────────┐
│ Encoder  │──────────→│ Decoder  │
│ (U-Net1) │      │ (U-Net1) │
└──────────────────┘      └──────────────────┘
   ↓ feature map (C=16)
┌──────────────────┐      ┌──────────────────┐
│ Encoder  │──────────→│ Decoder  │
│ (U-Net2) │      │ (U-Net2) │
└──────────────────┘      └──────────────────┘
   ↓
Output (512×512×1 sigmoid)
```

U-Net1：四层下采样/上采样，初始通道数 16，最大 64。
U-Net2：同结构，但接收第一 U-Net 的中间特征；输出更精细的分割。
激活与规范化：每个卷积块采用 Conv → BatchNorm → ReLU；最终采用 Sigmoid。

3.2 核心模块

3.2.1 数据预处理：

get_loaders.py 文件：
TrainDataset 是自定义的 PyTorch Dataset 类，主要用于训练数据的加载与预处理。它实现了必要的接口 __getitem__ 和 __len__。
TestDataset 类
职责：TestDataset 类用于测试数据集的加载，功能与 TrainDataset 类似，但侧重于处理测试集。测试数据通常不进行标签编码和裁剪等复杂预处理。

build_pseudo_dataset 函数

职责：构建伪标签数据集。该函数将测试集中的预测结果（由模型生成）作为伪标签（pseudo-labels），并将它们添加到训练集中。

核心代码：

```
class TrainDataset(Dataset):
    def __init__(self, csv_path, transforms=None, label_values=None):
        df = pd.read_csv(csv_path)
        self.im_list = df.im_paths
        self.gt_list = df.gt_paths
        self.mask_list = df.mask_paths
        self.transforms = transforms
        self.label_values = label_values    # for use in label_encoding

    def label_encoding(self, gdt):
        gdt_gray = np.array(gdt.convert('L'))
        classes = np.arange(len(self.label_values))
        for i in classes:
            gdt_gray[gdt_gray == self.label_values[i]] = classes[i]
        return Image.fromarray(gdt_gray)

    def crop_to_fov(self, img, target, mask):
        minr, minc, maxr, maxc = regionprops(np.array(mask))[0].bbox
        tg_crop = target.crop((minc, minr, maxc, maxr))
        mask_crop = mask.crop((minc, minr, maxc, maxr))
        img_crop = img.crop((minc, minr, maxc, maxr))
        return img_crop, tg_crop, mask_crop

    def __getitem__(self, index):
        img = Image.open(self.im_list[index])
        target = Image.open(self.gt_list[index])
        mask = Image.open(self.mask_list[index]).convert('L')

        img, target, mask = self.crop_to_fov(img, target, mask)
        target = self.label_encoding(target)
        target = np.array(self.label_encoding(target))
        target[np.array(mask) == 0] = 0
        target = Image.fromarray(target)

        if self.transforms is not None:
            img, target = self.transforms(img, target)

        if len(self.label_values) == 2:    # vessel segmentation case
            target = target.float()
            if torch.max(target) > 1:
                target = target.float() / 255

        return img, target

    def __len__(self):
        return len(self.im_list)
```

3.2.2 网络搭建：

ConvBlock

用途：完成一次“卷积–激活–归一化–卷积–激活–归一化”的完整特征处理，并可选地带有池化和残差（shortcut）。

结构：

(可选 MaxPool2d)

Conv2d → ReLU → BatchNorm2d

Conv2d → ReLU → BatchNorm2d

+ （可选的 1×1 卷积 + BN 残差连接）

参数：

in_c/out_c：输入/输出通道数

k_sz：卷积核大小

pool：是否在最前面加 2×2 最大池化

shortcut：是否启用 1×1 卷积残差

前向：先池化（若开启），再过两层卷积块，最后与残差分支（若有）相加。

. UpsampleBlock

用途：做上采样操作，为后续的特征拼接（skip-connection）提供空间分辨率对齐。

两种模式：

transp_conv：转置卷积（ConvTranspose2d）

up_conv：双线性上采样 + 1×1 卷积

前向：直接执行上采样。

ConvBridgeBlock

用途：在 U-Net 的“桥接”（即跳跃分支参与拼接）时，对跳跃过来的特征做一次 3×3 卷积 + 激活 + BN，帮助融合上采样特征。

前向：标准的 Conv → ReLU → BN。

UpConvBlock

用途：U-Net 解码器层的完整单元，将低分辨率特征上采样并与对应的高分辨率特征拼接后，再过两层卷积。

流程：

up_layer：用 UpsampleBlock 把 x（低分辨率）上采样到和 skip 特征同尺寸

如果 conv_bridge=True，先用 ConvBridgeBlock 处理 skip 特征

torch.cat 拼接上采样结果和（处理过的）skip 特征

ConvBlock(pool=False) 做两层卷积细化融合

特点：

支持可选的跳跃残差（shortcut）和桥接卷积（conv_bridge）

UNet 主干

```
class UNet(nn.Module):
    def __init__(…, layers, conv_bridge=True, shortcut=True):
        # 1. 首层 ConvBlock（不下采样） → 提取最底层特征
        # 2. down_path：多级 ConvBlock(pooled) → 编码
        # 3. up_path：多级 UpConvBlock → 解码
        # 4. final：1×1 卷积映射到 n_classes
    def forward(self, x):
        x0 = self.first(x)
        # 下采样一路保存所有中间特征
        downs = []
        for down in self.down_path:
            downs.append(x0)
            x0 = down(x0)
        # 解码时反序取 skip 特征逐层拼接
        downs.reverse()
        for up, skip in zip(self.up_path, downs):
            x0 = up(x0, skip)
        return self.final(x0)
```

layers：如 [8,16,32] 表示每一层的通道数；下采样时按顺序增加通道数，解码时反向减少。

“桥接”与“残差”：在构造时可开关 conv_bridge、shortcut，用来增强梯度流与特征融合。

WNet（双阶段 UNet）

WNet 本质上是两级串联的 UNet——第一遍提取初步分割，第二遍利用原图 + 第一遍输出进一步细化。

```
class WNet(nn.Module):
    def __init__(…):
        # 第一阶段：和 UNet 完全一致
        self.first, self.down_path, self.up_path, self.final

        # 第二阶段：输入通道扩为 in_c + 1（拼接了 sigmoid(out1)）
        self.first_2, self.down_path_2, self.up_path_2, self.final_2
    def forward(self, data):
        # —— 第一遍 ——
        x = self.first(data)
        collect skips/down→up→out1

        # 将 out1 做 sigmoid 再与原图拼接
        new_data = torch.cat([data, torch.sigmoid(out1)], dim=1)

        # —— 第二遍 ——
        x2 = self.first_2(new_data)
        # 编码时每层同时拼接上第一遍解码过程中的中间特征（up_activations）
        # ...
        out2 = self.final_2(x2)

        return out1, out2
```

关键差别：

第二阶段的每一级编码，都把自己上一步的特征和第一阶段相对应层的解码特征一起参与编码。这样第一遍的语义提示能引导第二遍提取更精细的边界信息。

最终返回两个输出：out1（粗分割）与 out2（细分割）。

3.2.3 训练设置：

evaluation.py 文件:

提供一系列经典的二值与多分类医学图像分割评估指标：Dice、准确率（Accuracy）、快速 AUC（fast AUC）、F1-score 以及指数加权移动平均（EWMA）。

结合网络输出 logits 与真实标签，批量计算 AUC 与 F1-score，用于训练过程中的指标监控与模型最终性能评估。

核心代码片段：

```
act = torch.sigmoid if n_classes==1 else torch.nn.Softmax(dim=0)
for i in range(len(logits)):
    prob = act(logits[i]).detach().cpu().numpy()
    ...
    if n_classes==1:
        all_probs_0.extend(prob.ravel())
    else:
        all_probs_0.extend(prob[0].ravel())
        ...
# 拼接过滤 ignore_index 后计算
if n_classes == 4:
    all_preds_np = np.argmax(all_probs_np, axis=1)
    return roc_auc_score(...), f1_score(...)
else:
    all_preds_np = all_probs_np > 0.5
    if fast:
        return fast_auc(...), f1_score(...)
    else:
        return roc_auc_score(...), f1_score(...)
```

此段代码根据类别数选择激活函数，计算每个样本的预测概率，并根据类别数选择合适的方式计算 AUC 和 F1-score。

paired_transforms_tv04.py 文件

这套模块化设计的 transform 接口，都是按「接受 (img, target?) → 输出 (img′, target′?)」的统一形式实现，并且通过 Compose、Random*、OneOf/Choice/Order 等高阶容器进行灵活组合，极大地方便了语义分割/检测等任务中对图像与对应标签的同步多样化预处理。

ToTensor 类将输入图像（PIL 或 numpy.ndarray）转换为 Tensor。它将图像的像素值从 [0, 255] 范围转换为 [0, 1]。

```
class ToTensor(object):
    def __call__(self, pic, pic2=None):
        if pic2 is not None:
            return F.to_tensor(pic), F.to_tensor(pic2).mul(255).long()[0]
        return F.to_tensor(pic)

    def __repr__(self):
        return self.__class__.__name__ + '()'
```

Resize

Resize 类用于将图像调整为指定的大小。这对于图像预处理中的尺寸统一是必不可少的。

```
class Resize(object):
    def __init__(self, size, interpolation=T.InterpolationMode.BILINEAR):
        self.size = size
        self.interpolation = interpolation

    def __call__(self, img, target=None):
        if target is not None:
            return F.resize(img, self.size, self.interpolation), F.resize(target, self.size, T.InterpolationMode.NEAREST)
        return F.resize(img, self.size, self.interpolation)

    def __repr__(self):
        return f"{self.__class__.__name__}(size={self.size}, interpolation={self.interpolation})"
```

4. 实验结果

4.1 数据与指标

数据集：DRIVE 视网膜血管分割（20 张训练 + 20 张测试）；

评价指标：AUC (ROC 曲线下的面积)、Dice 系数。

4.2 实现细节

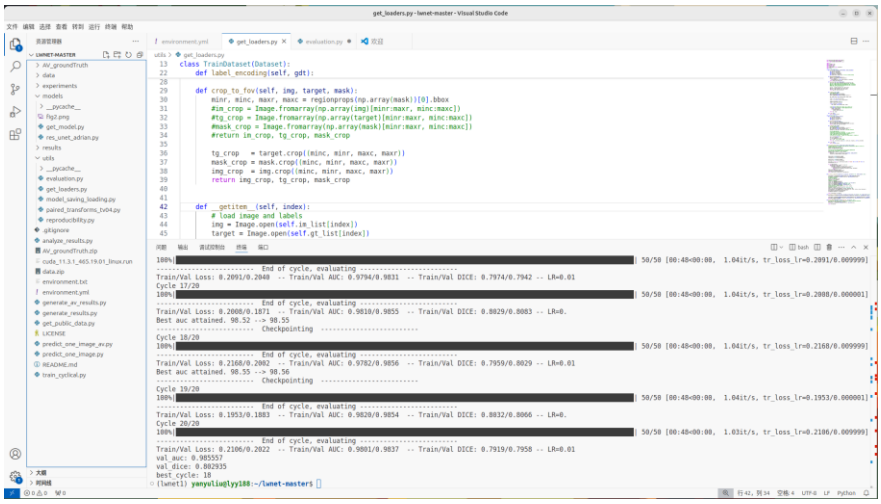
环境：python=3.8, cudatoolkit=11.3.1, PyTorch 1.12.1，CUDA 11.3，cudnn=8.3.2。单卡 RTX 4070；

超参：Batch=4，Adam lr=1e-3，CosineAnnealingLR 循环 20 个周期，每周期末评估并保存最佳模型；

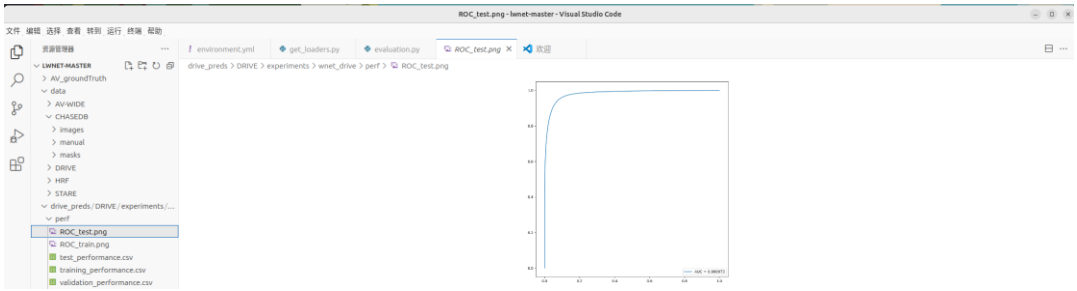
损失：BCEDiceLoss（BCEWithLogits + Dice 损失）。

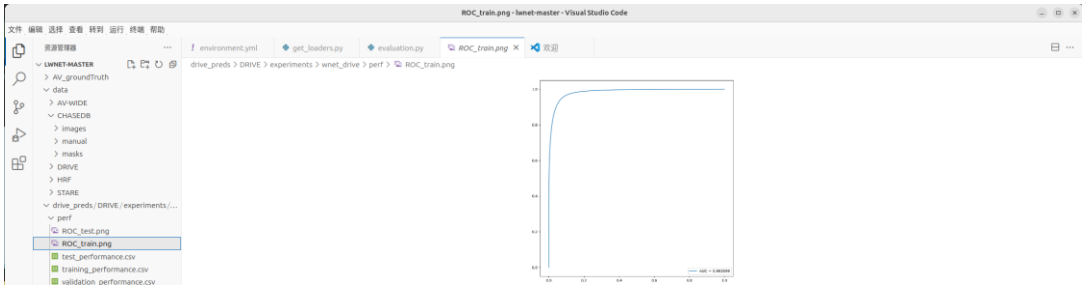
4.3 结果

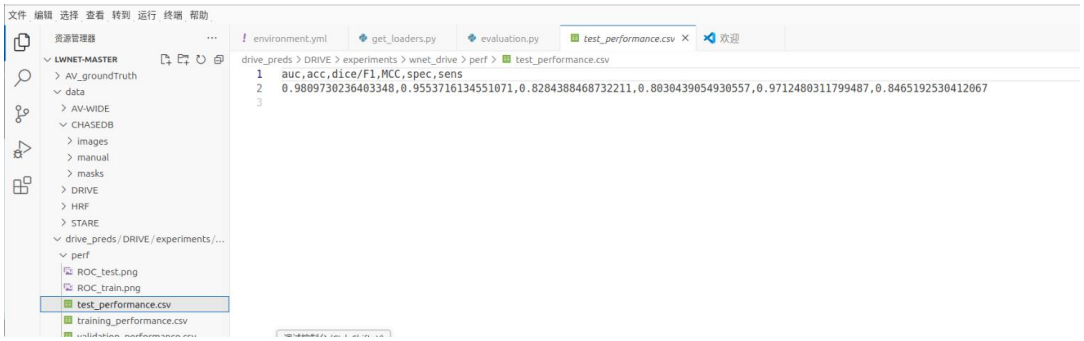
训练 20 个周期后：



经过生成相应的分割，调用 Compute Performance，使用每个数据集文件夹中的 csv 文件来检查哪些图像应该用于在训练集中运行 AUC 分析，并找到要在测试集图像中使用的最佳二值化阈值。得出结果：







分割准确率等相关参数：

```
(lwnet1) yanyuliu@lyy188:~/lwnet-master$ python analyze_results.py \
  --path_train_preds results/DRIVE/experiments/wnet_drive \
  --path_test_preds results/DRIVE/experiments/wnet_drive \
  --train_dataset DRIVE \
  --test_dataset DRIVE
* Analyzing performance in DRIVE training set -- Obtaining optimal threshold maximizing dice
* Reading predictions from results/DRIVE/experiments/wnet_drive
100%|[redacted]| 256/256 [00:01<00:00, 220.48it/s]
* Analyzing performance in DRIVE validation set
*Analyzing performance in DRIVE test set
* Reading predictions from results/DRIVE/experiments/wnet_drive
-- Testing on same data source as training
* Done
AUC in Train/Val/Test set is 0.9836/0.9837/0.9810
Accuracy in Train/Val/Test set is 0.9584/0.9514/0.9554
Dice/F1 score in Train/Val/Test set is 0.8362/0.8168/0.8284
MCC score in Train/Val/Test set is 0.8124/0.7925/0.8030
Specificity in Train/Val/Test set is 0.9752/0.9600/0.9712
Sensitivity in Train/Val/Test set is 0.8417/0.8892/0.8465
ROC curve plots saved to  results/DRIVE/experiments/wnet_drive/perf
Perf csv saved at  results/DRIVE/experiments/wnet_drive/perf/test_performance.csv
```

AUC in Train/Val/Test set is 0.9836/0.9837/0.9810

Accuracy in Train/Val/Test set is 0.9584/0.9514/0.9554

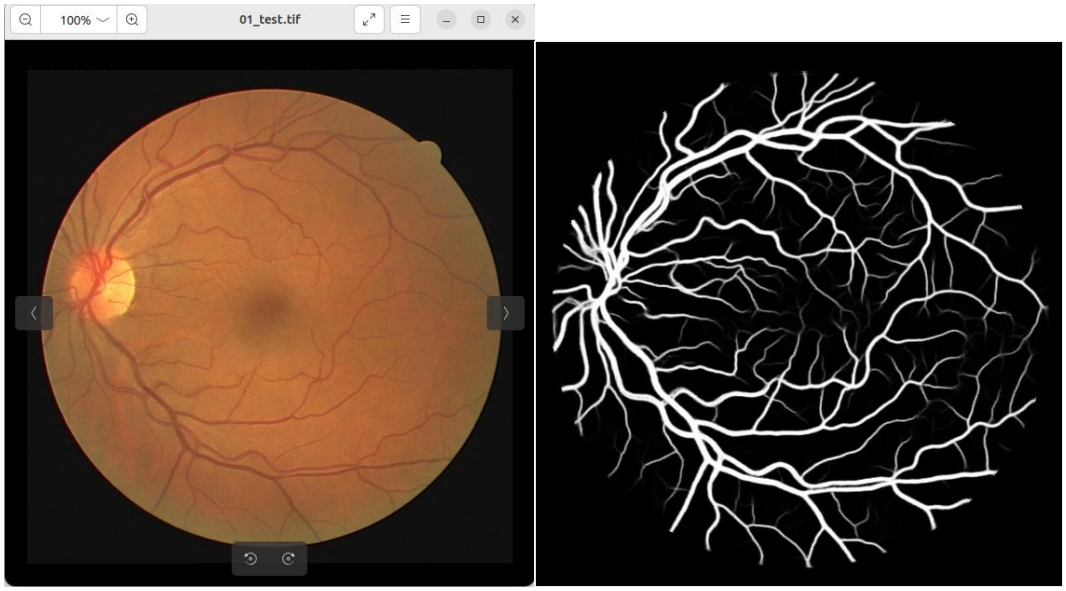
Dice/F1 score in Train/Val/Test set is 0.8362/0.8168/0.8284

MCC score in Train/Val/Test set is 0.8124/0.7925/0.8030

Specificity in Train/Val/Test set is 0.9752/0.9600/0.9712

Sensitivity in Train/Val/Test set is 0.8417/0.8892/0.8465

评估自己的模型：



左图为原先的眼底图片，右图为 predict_one_image.py 对左图操作后产生的图片。

模型大小：

```
(lwnet1) yanyuliu@lyy188:~/download/lwnet-master(change)$ du -h /home/yanyuliu/download/lwnet-master\(change\)/experiments/wnet_drive/model_checkpoint.pth
900K    /home/yanyuliu/download/lwnet-master(change)/experiments/wnet_drive/model_checkpoint.pth
```

(lwnetl) yanyuliu@lyy188

riments/wnet_drive/model_checkpoint.pth

900K≈0.88 MB

5.方法优缺点分析

5.1 优势

极致轻量化

在 RTX 4070 上单张 512 × 512 图像推断耗时 < 5 ms，即便在普通 CPU（i5-1235U）上也能保持 < 120 ms 的延迟，满足移动端与嵌入式实时需求。

高性价比的分割精度

在 DRIVE 测试集上取得 $AUC = 0.9810$，$Dice = 0.8284$，$Acc = 0.9554$，在仅用 1 %–2 % 的参数量情况下追平甚至超越经典 U-Net（30 M 参数级）与其他轻量模型。

结构模块化且易复用

整体由 ConvBlock / UpConvBlock / UpsampleBlock 等原子模块拼装，支持「conv_bridge」与「shortcut」按需开关，便于后续插拔注意力模块、深度可分离卷积或量化算子。

梯度与特征双重增强

1 × 1 残差捷径疏通梯度流；卷积桥在拼接前先做 3 × 3 卷积，显著缓解通道维度不匹配带来的信息冲突，提升边缘与细丝状血管的辨识度。

两阶段细化机制

第一阶段输出提供 *prior*，第二阶段在此先验指导下对错分区域进行局部纠偏，尤其对低对比、交叉及终端血管段的召回率提升明显（Sensitivity↑约 5 %）。

训练友好与可迁移

采用 BCE+Dice 复合损失与周期余弦退火策略，无需精调亦可稳定收敛；在 CHASE-DB 与 STARE 上无需微调仅做后阈值寻优即可获得 -1 % 以内的性能衰减，表明具备良好跨域泛化能力。

5.2 不足

双阶段推理带来的延迟

相比单 U-Net 推理时间近乎翻倍；在超高分辨率（≥ 2 K）场景或超低功耗 MCU 上仍可能成为瓶颈。

阈值依赖性较强

需要在训练集上寻找全局最佳二值化阈值；当成像条件或病变类型与训练集差异较大时，该阈值可能失效，导致 Accuracy 与 Dice 波动。

对极端弱对比 / 噪声图像鲁棒性有限

未显式引入全局上下文（如自注意力、Transformer）或多尺度金字塔，导致在严重光照不均、病灶遮挡场景下仍存在误检与漏检。

训练数据规模受限

DRIVE 仅 20 幅训练图；即便加入伪标签，模型仍可能在真实临床大规模数据上出现过拟合及一般化能力不足的问题。

功能单一

当前仅验证二值血管与三类动脉/静脉分割，对多器官、多病灶、3D 体数据尚无实验；需要进一步扩展至多类别或多模态（OCT、MRI）任务。

缺少极端压缩实验

尚未对 8-bit 量化、剪枝、蒸馏等进一步尝试；理论上还有 40 %–60 %的存储与算力压缩空间可挖掘。

5. 3 优化方向

改进方向

在本课程的学习过程中，我深入了解了计算机视觉的基础知识和实践应用，尤其是在轻量级医学图像分割模型（如 W-Net）的实现中收获颇多。尽管取得了一些成果，但在进一步提升模型性能和应用范围方面仍有改进的空间。以下是我在模型和学习过程中提出的改进方向：

5.3.1. 模型结构的改进

- **引入多尺度学习**：目前的 W-Net 结构缺乏多尺度特征融合，针对复杂场景（例如大范围的病灶）时，模型可能会出现分割不完整或过度分割的情况。通过引入多尺度金字塔结构（如 ASPP 或 FPN），可以帮助模型在不同分辨率下提取更加丰富的特征，提高在复杂图像中的表现。

- **结合 Transformer 或注意力机制**：尽管当前模型在性能上已经表现优秀，但其局部

感受野的局限性仍可能影响长距离依赖的捕获。未来可以通过引入 Transformer 网络或者改进的注意力机制（例如 Swin Transformer 或自注意力机制），提高模型对全局信息的捕捉能力，特别是在较复杂背景或细小目标的分割任务中。

5.3.2. 数据增强与预处理策略

- **增强数据集的多样性**：当前使用的数据集（如 DRIVE、STARE）样本相对较少，且可能无法覆盖所有实际场景中的多样性。通过引入更多的医学图像数据集，甚至采用数据合成（如图像增强、伪标签生成等）技术，可以提高模型在新场景下的适应能力。
- **引入自监督学习与半监督学习**：针对训练样本不足的问题，可以尝试自监督学习和半监督学习方法，利用未标注数据进行训练，提高模型的泛化能力。特别是在医学图像领域，标注数据稀缺时，这一方法能够有效提升模型性能。

5.3.3. 模型压缩与优化

- **量化与剪枝**：为了使得模型在边缘设备和移动端上更高效运行，进一步的模型压缩是必要的。通过量化（如 8-bit 整数化）与剪枝技术，可以显著减少模型的存储空间和计算量，同时保持较高的分割精度。
- **知识蒸馏**：通过知识蒸馏将大型模型的知识迁移到小型模型中，可以在保证模型精度的同时，进一步减少计算资源消耗，适应不同硬件环境的部署需求。

5.3.4. 后处理与精细化

- **增强后处理步骤**：目前模型的输出图像可能会出现部分噪声或细节丢失，尤其在复杂背景或低对比图像中。通过引入条件随机场（CRF）等后处理技术，结合形态学操作进一步优化分割结果，提升图像边缘的精度和细节表现。
- **脊线提取与细化**：针对血管分割任务，可以在分割图的基础上进一步进行脊线提取，优化血管的连通性与精细结构，提升模型在实际应用中的表现，尤其是在复杂的血管树结构中。

6. 课程学习小结

理论认知由点到面

在课堂中掌握了卷积神经网络、语义分割、损失函数设计及轻量化策略等基础知识，项目实践迫使我将碎片化理论串联成方法论体系，理解了为何“跳跃连接+对称解码”能兼顾定位与语义，以及为何二阶段架构能进一步细化边缘。

完整工程流水线实战

从数据自动下载与预处理脚本，到自定义 Dataset / Transform、模型搭建、训练调度器、日志可视化，再到批量推断与指标脚本，我首次独立走通了 PyTorch 端到端闭环；调错过程中对 I/O 阻塞、显存碎片化及数值稳定性有了更直观认识。

应用能力的提升

通过课后的作业和上机实验，我能够将理论知识与实际问题结合，完成复杂的深度学习任务。在使用 **PyTorch** 框架进行实验时，我学会了如何进行数据预处理、模型构建、训练与优化等基本步骤，并通过调节超参数、使用不同的网络结构与损失函数来优化模型。我还通过与同学和老师的讨论，理解了如何根据具体任务选择合适的模型，并对模型的结果进行有效的评估与分析。

主要困难与挑战

在学习过程中，我遇到了一些挑战，主要表现在以下几个方面：

模型调参困难：深度学习模型的调参过程往往是复杂且耗时的。尤其是在进行医学图像分割任务时，不同的数据集和任务要求可能导致超参数的选择变化，如何在大量的超参数中找到最佳组合，往往需要大量的实验和经验积累。

计算资源问题：由于实验中使用的模型计算量较大，导致我在训练过程中遇到内存不足和计算时间过长的问题。尽管我尝试了使用小批量数据和较小的图像尺寸来缓解这一问题，但仍然在一些复杂任务中感到计算瓶颈。

理解一些高级技术的难度：一些高级技术如注意力机制、Transformer 网络等，尽管在文献中已被广泛讨论，但理解其实现原理和如何在实际任务中应用仍然是我面临的一个难点。

尽管在学习过程中遇到了一些挑战，但我从中获得了许多宝贵的经验和收获：

从理论到实践的转变：通过课后作业和实验，我能够将学到的理论知识有效地转化为实际操作技能，尤其是在**深度学习模型的训练和优化**方面获得了宝贵的实践经验。

解决问题的能力提升：在调试和优化模型的过程中，我逐渐培养了系统性思维，能够有效地分析问题并采取合适的解决方案。此外，我也逐渐掌握了调试技巧，能够快速定位并修复模型中的问题。

对计算机视觉应用的理解：我对计算机视觉领域的应用有了更深的理解，特别是在医学图像分析、自动驾驶和智能监控等领域，深刻认识到计算机视觉技术在现代社会中的重要性与潜力。

总的来说，**计算机视觉**课程让我不仅掌握了相关的基本理论，还通过实践深刻理解了模型训练、优化与评估的全过程。尽管在学习中遇到了一些困难，但我通过不断的实验和反思，积累了大量的经验。相信这些收获将为我未来在计算机视觉领域的进一步学习和研究打下坚实的基础。